Operating Systems Lab 9

Synchronization – Software and Hardware Solutions

Lab Outline

By the end of this lab, students will be able to:

- Understand the concept of synchronization.
- Identify race conditions and critical sections in a program.
- Understand why synchronization is required when multiple threads share data.
- Learn a software-based solution (Peterson's Solution).
- Learn a hardware-based solution (Compare-and-Set).
- Apply synchronization concepts to solve simple multithreading problems.

Introduction

In modern operating systems, multiple threads often execute concurrently and may need to access the same shared data. While concurrent execution improves performance, it can also create problems when multiple threads attempt to modify the same variable simultaneously.

Consider a shared variable named `count`.

Suppose one thread continuously increments the value of `count` while another thread continuously decrements it. Logically, if both operations are performed an equal number of times, the final value should remain unchanged.

However, when the program is executed, the final value is often different from the expected result. This happens because both threads are accessing and modifying the same variable at the same time.

This problem is known as a **Race Condition**.

The portion of code where shared data is accessed or modified is called a **Critical Section**.

For example:

```
count++;
```

and

```
count--;
```

are critical sections because they modify a shared variable.

The goal of synchronization is to ensure that only one thread accesses the critical section at a time so that shared data remains consistent and correct.

Why Does the Problem Occur?

Many students assume that:

```
count++;
```

is a single operation.

In reality, the processor performs several steps:

1. Read the value of `count` from memory.
2. Increment the value.
3. Write the updated value back to memory.

If another thread modifies the same variable between these steps, the result becomes incorrect.

This is why synchronization mechanisms are required.

Peterson's Solution (Software Solution)

Peterson's Solution is a classical software-based solution to the critical section problem.

It works for two processes or two threads and uses two shared variables:

```
flag[2];
turn;
```

The `flag` array indicates whether a thread wants to enter the critical section.

The `turn` variable determines which thread gets the chance to enter first when both threads request access simultaneously.

Before entering the critical section, a thread announces its intention by setting its flag and then waits if the other thread also wants to enter.

This guarantees:

- Mutual Exclusion

- Progress
- Bounded Waiting

Although Peterson's Solution is important for understanding synchronization concepts, modern processors may reorder memory operations, making the solution unreliable on many real systems without additional memory barriers.

Hardware Solution – Compare and Set

Modern processors provide hardware instructions that perform certain operations atomically.

One such instruction is **Compare-and-Set (CAS)**.

The instruction checks whether a variable contains an expected value. If it does, the variable is updated atomically.

Because the entire operation occurs as a single indivisible step, no other thread can interfere during execution.

This makes Compare-and-Set an effective hardware solution for implementing locks and synchronization primitives.

Most mutexes and semaphores provided by operating systems internally rely on such hardware-supported atomic instructions.

Example 1

Race Condition without Synchronization

Scenario

Suppose a bank account balance is stored in a shared variable.

One thread deposits money while another thread withdraws money.

Both operations occur simultaneously.

Program

```
#include <stdio.h>
#include <pthread.h>

long balance = 1000;
```

```
void* deposit(void* arg)
{
    for(int i=0;i<500000;i++)
        balance++;

    return NULL;
}

void* withdraw(void* arg)
{
    for(int i=0;i<500000;i++)
        balance--;

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    printf("Initial Balance = %ld\n", balance);

    pthread_create(&t1, NULL, deposit, NULL);
    pthread_create(&t2, NULL, withdraw, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Balance = %ld\n", balance);

    return 0;
}
```

Discussion

The expected final balance should remain 1000 because deposits and withdrawals are equal.

However, the result is often incorrect due to a race condition.

This demonstrates the need for synchronization.

Example 2

Synchronization using Compare-and-Set

Scenario

Suppose two workers are updating the same inventory count.

Only one worker should modify the inventory at a time.

Program

```
#include <stdio.h>
#include <pthread.h>

long compare_and_set(long *flag, long expected, long new);

asm(
"compare_and_set:;"
"movq %rsi,%rax;"
"lock cmpxchgq %rdx,(%rdi);"
"retq;"
);

long inventory = 100;
long lockFlag = 0;

void* increase(void* arg)
{
    for(int i=0;i<1000000;i++)
    {
        while(compare_and_set(&lockFlag,0,1)!=0);

        inventory++;

        lockFlag = 0;
    }

    return NULL;
}

void* decrease(void* arg)
{
    for(int i=0;i<1000000;i++)
    {
        while(compare_and_set(&lockFlag,0,1)!=0);

        inventory--;

        lockFlag = 0;
    }

    return NULL;
}

int main()
{
    pthread_t t1,t2;

    printf("Initial Inventory = %ld\n", inventory);

    pthread_create(&t1,NULL,increase,NULL);
    pthread_create(&t2,NULL,decrease,NULL);
```

```
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);

    printf("Final Inventory = %ld\n", inventory);

    return 0;
}
```

Discussion

The Compare-and-Set instruction acts as a lock.

Before entering the critical section, a thread acquires the lock.

Other threads must wait until the lock is released.

As a result, only one thread modifies the shared variable at a time, producing the correct final result.

Conclusion

Synchronization is necessary whenever multiple threads access shared data.

Without synchronization, race conditions may occur and produce incorrect results.

A critical section is the portion of code that accesses shared resources.

Peterson's Solution provides a software-based approach to solving the critical section problem, while Compare-and-Set provides a hardware-supported atomic solution.

Understanding these concepts forms the foundation for advanced synchronization mechanisms such as mutexes, semaphores, and monitors used in modern operating systems.

Practice Task 1

Scenario

A university library maintains a shared variable named `booksAvailable` that stores the number of available books.

Two threads are created:

- Thread 1 issues books to students by decrementing `booksAvailable`.
- Thread 2 receives returned books and increments `booksAvailable`.

Both threads perform their operations 500,000 times.

Task

Write a C program using **POSIX threads (pthreads)** that:

1. Creates two threads.
2. Uses a shared variable `booksAvailable`.
3. One thread increments the variable.
4. The other thread decrements the variable.
5. Uses the **Compare-and-Set hardware solution** to synchronize access to the shared variable.
6. Displays the initial and final value of `booksAvailable`.

Practice Task 2

Scenario

A company stores employee salaries in an array of 1000 elements.

To calculate the total payroll quickly, the company divides the work among multiple threads.

Task

Write a C program that:

1. Creates an integer array of 1000 elements.
2. Initializes all elements with random values.
3. Creates **5 threads**.
4. Each thread calculates the sum of 200 elements.
5. All threads add their partial sums to a shared variable `totalSalary`.
6. Uses a hardware lock (Compare-and-Set) to ensure that only one thread updates `totalSalary` at a time.
7. Displays the final sum.